

ASSIGNMENT 1: DEADLINE PANIC & TASK MANAGER ADTs

Goal: The goal of this assignment is to get some practice with queue, stack, and doubly linked list implementation. One of the goals is also to explore the use of these data structures in the context of an application. In Part (a) of the assignment you have to implement the foundational abstract data types that power a Task Manager. In Part (b) of the assignment you have to wire these ADTs into a Task Manager engine and solve a puzzle.

Language & Constraints: Your code must compile and run with a standards-compliant C++17 compiler (g++ recommended). You must **not** use the Standard Template Library containers `<vector>`, `<queue>`, `<stack>`, or `<list>` (nor any equivalent container from `<deque>`, `<forward_list>`, etc.). You may use `<string>`, `<iostream>`, `<fstream>`, and `<stdexcept>` as needed. All nodes must be dynamically allocated using `new`, and every class that owns heap memory must provide a destructor that frees all allocated memory when the object goes out of scope. All basic operations (`push`, `pop`, `enqueue`, `dequeue`, `prepend`, `append`) must run in strictly $O(1)$ time complexity.

PART 1(A): FOUNDATIONAL ABSTRACT DATA TYPES

Problem Statement: Your first task is to implement the underlying data structures for a Task Manager, creating a separate class for each ADT. The exact interfaces will be as follows.

Task

A Task object stores the information associated with a single to-do item:

```
class Task {
public:
    int taskId;
    std::string description;
    int priority;    // 0 - standard, 1 - urgent, 2 - emergency

    Task();
    Task(int taskId, const std::string& description, int priority);
    std::string toString() const;
};
```

Priority values are restricted to $\{0, 1, 2\}$ as indicated above.

You should provide a `toString` method that returns a single-line human-readable summary of the task. A task with `taskId = 1`, `description = "Task-1"`, and `priority = 2` should return exactly `Task(id=1, desc="Task-1", priority=2)`. There must be no leading or trailing whitespace, and the field order must be `id`, then `desc`, then `priority` as shown.

TaskQueue (buffer)

A standard queue that holds incoming tasks before they are scheduled:

```

class TaskQueue {
public:
    TaskQueue();
    ~TaskQueue();

    void enqueue(const Task& task);           // append
    void insertFirst(const Task& task);       // prepend
    Task dequeue();                          // throws EmptyQueueException
    const Task& front() const;               // throws EmptyQueueException
    bool isEmpty() const;
    int size() const;
    std::string toString() const;
};

```

Your implementation must be done in terms of a singly linked list of dynamically allocated nodes, with pointers that allow **enqueue**, **insertFirst** and **dequeue** in $O(1)$ time.

You should provide a **toString** method in addition to the other queue methods. **toString** must return the task ids currently in the queue, listed from front to rear, separated by a comma and a single space, and enclosed in square brackets. An empty queue should return []. A queue into which tasks with ids 1, 2, and 3 were enqueued in that order should return [1, 2, 3]. Do **not** include descriptions or priorities in this string — task ids only.

TaskStack (undo history)

A stack that stores deleted tasks so they can be restored by an undo feature:

```

class TaskStack {
public:
    TaskStack();
    ~TaskStack();

    void push(const Task& task);
    Task pop();                          // throws EmptyStackException
    const Task& top() const;              // throws EmptyStackException
    bool isEmpty() const;
    int size() const;
    std::string toString() const;
};

```

Your implementation must be done in terms of a singly linked list of dynamically allocated nodes, with **push** and **pop** in $O(1)$ time.

You should provide a **toString** method in addition to the other stack methods. **toString** must return the task ids currently on the stack, listed from top to bottom, separated by a comma and a single space, and enclosed in square brackets. An empty stack should return []. A stack onto which tasks with ids 1, 2, and 3 were pushed in that order should return [3, 2, 1] (top appears first). Do **not** include descriptions or priorities in this string — task ids only.

TaskTimeline (the main schedule)

A doubly linked list that represents the chronological timeline of scheduled tasks:

```

class TaskTimeline {
public:
    TaskTimeline();
    ~TaskTimeline();

```

```

void insertFirst(const Task& task);    // prepend, O(1)
void insertLast(const Task& task);    // append, O(1)
Task removeFirst();                  // removes and returns front task,
                                     // throws if empty
bool remove(int taskId);              // true if removed
Task* find(int taskId);               // nullptr if absent
void print() const;                  // writes toString() followed by a newline
std::string toString() const;
bool isEmpty() const;
int size() const;
};

```

`insertFirst`, `insertLast` and `removeFirst` must run in $O(1)$ time. `remove` and `find` may scan the list and are permitted to take $O(n)$ time.

You should provide a `toString` method in addition to the other timeline methods. `toString` must return the task ids currently on the timeline, listed from head to tail (chronological order), separated by a comma and a single space, and enclosed in square brackets. An empty timeline should return `[]`. A timeline containing tasks with ids 1, 2, and 3 in chronological order should return `[1, 2, 3]`. Do **not** include descriptions or priorities in this string — task ids only. The `print` method must write exactly `toString()` followed by a newline to standard output.

Ensure that the version of your code you hand in does not produce any extraneous debugging output anymore!

What is being provided? Your assignment packet contains two folders with the starter code to each of the assignment parts. **part_A** of the packet contains the following files —

1. `EmptyQueueException.h`: Custom exception for empty-queue errors. (*You should not modify this file*)
2. `EmptyStackException.h`: Custom exception for empty-stack errors. (*You should not modify this file*)
3. `Task.h` / `Task.cpp`: Task class declaration and stub. Implement the constructors and `toString`.
4. `TaskQueue.h` / `TaskQueue.cpp`: Here you have to implement all the functions in the `TaskQueue` class.
5. `TaskStack.h` / `TaskStack.cpp`: Here you have to implement all the functions in the `TaskStack` class.
6. `TaskTimeline.h` / `TaskTimeline.cpp`: Here you have to implement all the functions in the `TaskTimeline` class.
7. `PartATester.cpp`: File with the `main` function that exercises the ADTs you will implement. If you run this code without any changes, it raises `std::runtime_error: Not implemented yet`. (*You should not modify this file*)

How to run? Assignment packet **part_A** contains a `Makefile` with the necessary commands to build and run the program. If you are unable to run the makefile, just copy-paste the commands in the makefile `all:` section sequentially in your shell.

PART 1(B): DEADLINE PANIC

Problem Statement: It is midsem week, and your life has officially declared war on your calendar. Assignment deadlines, quiz reminders, laundry you swore you would do on Monday,

and that one club meeting you RSVP'd to in a moment of optimism — they are all arriving faster than you can handle. Your roommate took one look at your sticky-note wall and said, “*Maybe try software?*”

So you built a Task Manager. It works like this:

- Incoming tasks have priorities (0, 1, or 2) that dictate how they are handled.
 - Priority 0 (**Standard**): Tasks land at the rear of your **inbox** — a **TaskQueue**
 - Priority 1 (**Urgent**): Tasks skip the line and land at the front of your inbox (TaskQueue), but still wait to be processed.
 - Priority 2 (**Emergency**): Tasks bypass the inbox entirely and violently interrupt your schedule, landing directly at the front of your **schedule** — a **TaskTimeline**
- When you feel (briefly) productive, you **process** the next inbox task, appending it to the rear of your **schedule** — a **TaskTimeline** doubly linked list that stores tasks in the order you accepted them.
- Sometimes you stare at a scheduled task and decide *absolutely not*. You **delete** it. Being you, you keep deleted tasks in an undo **history** — a **TaskStack** — because regret arrives on schedule too.
- Sometimes you actually finish something: you **complete** a task, and it is removed from the timeline for good (no undo — celebrate responsibly). You can choose to complete a specific task by its ID, or simply complete whatever is currently at the front of your schedule.
- And at 2 a.m., naturally, you **undo** the last deletion and shove that task back onto the end of your timeline.

Your goal is to programmatically simulate this Task Manager over a sequence of operations and report what happens on every action that can succeed or fail.

Input: The input to your software is a sequence of operations. The first line is the total number of operations N . Every successive line is operation $\#i$ with i ranging from 1 to N . Each operation begins with an action id:

Id	Action	Additional arguments
1	RECEIVE	taskId priority
2	PROCESS	—
3	DELETE	taskId
4	COMPLETE	[taskId](optional)
5	UNDO	—
6	PRINT	—

For **RECEIVE**, create a task whose description is “Task-<taskId>” (for example, task id 42 has description “Task-42”). Priority is an integer in $\{0, 1, 2\}$. **RECEIVE** handles the task routing based on the 0, 1, or 2 priority logic described above. **PROCESS** dequeues the front inbox task and appends it to the timeline. **DELETE** removes the given task from the timeline and pushes it onto the undo stack. **COMPLETE** removes the given task from the timeline permanently (it is *not* pushed onto the undo stack). If a taskId is provided, it removes that specific task; if no taskId is provided, it removes the task at the front of the timeline. **UNDO** pops the most recently deleted task from the undo stack and appends it back onto the timeline. **PRINT** asks you to report the current timeline.

As an example, suppose you perform the following 10 operations: receive task 101 with priority 2, receive task 102 with priority 1, process, process, delete 101, undo, receive task 103 with priority 0, process, complete 102, print. The input will be:

```

10
1 101 2
1 102 1
1 103 0
2
3 101
5
2
4 102
4
6

```

Output: Produce one output line for every operation whose action id is in {2, 3, 4, 5, 6} (that is, every operation except **RECEIVE**). Each such line begins with the operation number, followed by a space, followed by the result:

- **PROCESS:** print the `taskId` that was moved to the timeline; if the inbox is empty, print `-1`.
- **DELETE:** print the `taskId` that was deleted; if it is not on the timeline, print `-1`.
- **COMPLETE:** print the `taskId` that was completed; if it is not on the timeline, (or if the timeline is empty for a front-completion), print `-1`.
- **UNDO:** print the `taskId` that was restored; if the undo stack is empty, print `-1`.
- **PRINT:** print the timeline using the same `toString` format as Part (A), e.g. `[101, 103]` or `[]` if empty.

In the example above, the output will be:

```

4 102
5 101
6 101
7 103
8 102
9 101
10 [103]

```

Explanation: 101 is Priority 2, so it jumps straight to the timeline. 102 is Priority 1, so it goes to the front of the queue. 103 is Priority 0, so it goes to the back of the queue. The first **PROCESS** moves 102 to the timeline `[101, 102]`. **DELETE** 101 moves it to the stack. **UNDO** restores 101 to the back of the timeline `[102, 101]`. The second **PROCESS** moves 103 to the timeline `[102, 101, 103]`. **COMPLETE** 102 removes it specifically `[101, 103]`. The parameter-less **COMPLETE** removes the front task, 101, leaving `[103]`.

Code: Your code must compile and run with C++17. Your code will be run with a parameter `inputfile`, which will have the input, and your code should output one line per non-**RECEIVE** action as per the output format above. You must implement a `TaskManager` class that acts as the controller for the system. It should instantiate **one** `TaskQueue`, **one** `TaskTimeline`, and **one** `TaskStack`. The interface should be:

```

class TaskManager {
public:
    TaskManager();
    ~TaskManager();

    void receiveTask(const Task& task);

```

```

int processNextTask();    // returns taskId, or -1 if inbox empty
int deleteTask(int taskId); // returns taskId, or -1 if absent
int completeTask(int taskId); // returns taskId, or -1 if absent
int completeTask();      // returns front taskId, or -1 if timeline empty
int undo();              // returns restored taskId, or -1 if none
std::string printTimeline() const;
};

```

Your implementation of **TaskManager** **MUST** use the **TaskQueue**, **TaskStack**, and **TaskTimeline** implementations from Part (A) of the assignment. Do **NOT** use any other data structure like arrays-as-containers or any STL container to implement these ADTs. The driver program that reads the input file and invokes **TaskManager** should live in **DeadlinePanic.cpp**.

Desiderata: **receiveTask**, successful **processNextTask**, **undo** (when the stack is non-empty), front-based **completeTask** and timeline append/prepend must each run in $O(1)$ time. **deleteTask** / ID-based **completeTask** may take $O(n)$ time to locate the task on the timeline.

What is being provided? **part.B** of your assignment packet contains the following files —

1. **TaskManager.h** / **TaskManager.cpp**: Here you have to implement all the functions in the **TaskManager** class.
2. **TaskManagerTester.cpp**: File with the **main** function that tests the **TaskManager** methods you will implement. If you run this code without any changes, it raises `std::runtime_error: Not implemented yet.` (*You should not modify this file*)
3. **DeadlinePanic.cpp**: File with the **main** function to implement the program that solves this puzzle.

Your assignment packet also contains a **formatChecker.py**, **sampleInputFile.txt**, and **sampleOutputFile.txt**. Running `python3 formatChecker.py sampleInputFile.txt sampleOutputFile.txt` double checks whether your code satisfies the input-output requirements of the assignment. Note that it does not check the correctness of your code but just confirms the input / output format. Usage: `python3 formatChecker.py <inputfile> <outputfile>`

How to run? Assignment packet **part.B** contains a **Makefile** with the necessary commands to build and run the program. If you are unable to run the makefile, just copy-paste the commands in the makefile **all:** section sequentially in your shell.

Assignment Folder Structure

The starter code can be found here: [resources](#)

Assignment packet follows a certain folder structure, so here we provide some explanation for that. Within the **resources** directory you should see 2 folders: **part.A** and **part.B**. Once you run the makefile present in any of the part folders, all object / executable files are created in the third folder called **build**. This is done so that **part.B** can link against your **part.A** implementations. The **-I** and **-L/-o** flags in the **g++** commands are used for that purpose. You should read more about them for a better understanding.

What to submit?

1. You need to submit a zip file **<your-entry-number>.zip** which contains your code, structured as mentioned in the subsequent points.

2. Your submission upon extraction, should yield a folder named `<your-entry-number>`.
3. This folder should contain 2 folders named `"part_A"` and `"part_B"`, and a `"writeup.txt"` file.
4. The folder `"part_A"` contains the `.h/.cpp` files corresponding to your implementation of the foundational ADTs (`Task`, `TaskQueue`, `TaskStack`, `TaskTimeline`).
5. The folder `"part_B"` contains the `.h/.cpp` files corresponding to your implementation of `TaskManager` and `DeadlinePanic`.
6. The `"writeup.txt"` should have a line that lists names of all students you discussed/collaborated with (see guidelines on collaboration vs. cheating on the course home page). If you never discussed your assignment with anybody, write `"None"`. After this you are welcome to write any comments about your submission, though this is not necessary.
7. Eg. If your entry number is 2025AIB0001, then you need to submit a zip file 2025AIB0001.zip. This zip file on extraction gives a directory structure as follows:

```

2025AIB0001
|
----part_A
|   |
|   -- .h / .cpp files for the foundational ADTs
|
----part_B
|   |
|   -- .h / .cpp files for TaskManager & Deadline Panic
|
---- writeup.txt

```

8. Please compress your directory structure into a `.zip` file only.
9. Please submit the `.h` and `.cpp` source files only (no binaries, no `build/` artifacts).
10. You will be penalized for any submissions that do not conform to the above requirements.

Evaluation Criteria

The assignment is worth 6 points. Your code will be autograded against a series of tests. 3.5 points are for autograding and 2.5 points are for demo with the TA.

What is allowed? What is not?

1. This is an individual assignment.
2. Your code must be your own. You are not to take guidance from any general purpose code or problem specific code meant to solve these or related problems.
3. You are not allowed to use built-in (or anyone else's) implementations of stacks, queues, lists, and other basic data structures. A key aspect of the course is to have you learn how to implement these data structures. In particular, do **not** use `<vector>`, `<queue>`, `<stack>`, or `<list>`.

4. You should develop your algorithm using your own efforts. You should not Google search or use AI tools for direct solutions to this assignment. However, you are welcome to Google search for generic C++-related syntax.
5. You must not discuss this assignment with anyone outside the class. Make sure you mention the names in your write-up in case you discuss with anyone from within the class. Please read academic integrity guidelines on the course home page and follow them carefully.
6. Your submitted code will be automatically evaluated against another set of benchmark problems. You get a significant penalty if your output is not automatically parsable and does not follow input-guidelines.
7. We will run plagiarism detection software. Anyone found guilty will be awarded a suitable penalty as per course guidelines, and institute rules.
8. As per the LLM policy of the class, no code should be written an LLM. However, you are allowed to use an LLM to understand C++ syntax, and also to brainstorm ideas related to the assignment.